

LLM evaluation and observability

These topics are part of **LLM evaluation and observability** used in modern **RAG systems**. Tools like RAGAS, Arize Phoenix, and TruLens help analyze how well a RAG pipeline works.

1 RAGAS Evaluation

What it is

RAGAS evaluates the quality of a **Retrieval-Augmented Generation (RAG)** system.

Instead of manually checking answers, RAGAS **automatically scores your RAG pipeline**.

Typical workflow:

User Question



Retriever (Vector DB)



Context



LLM Answer



RAGAS Evaluation

What RAGAS Measures

RAGAS provides several important metrics.

Metric	Meaning
Context Precision	Are retrieved docs relevant?
Context Recall	Did the retriever fetch all needed info?
Faithfulness	Is the answer grounded in context?
Answer Relevance	Does the answer match the question?

Example result:

Context Precision: 0.82

Context Recall: 0.76

Faithfulness: 0.91

Answer Relevance: 0.88

Interpretation:

- Retriever quality
- Answer correctness
- Hallucination probability

Example Code

```
from ragas import evaluate
```

```
result = evaluate(dataset)
```

```
print(result)
```

This produces evaluation scores for your RAG system.

2 Hallucination Detection

What is Hallucination

Hallucination occurs when the LLM **generates information not present in the retrieved context**.

Example:

Context:

The waiting period is 30 days.

LLM answer:

The waiting period is 60 days.

This is a hallucination.

How Hallucination is Detected

There are several methods.

Method 1 — Context Verification

Check whether answer statements exist in retrieved context.

Answer sentence



Compare with context



If mismatch → hallucination

Method 2 — LLM-as-a-Judge

Another LLM evaluates the answer.

Example prompt:

Does the answer rely only on the given context?

Output:

Faithful / Not Faithful

This method is used by **RAGAS** and **Phoenix**.

Method 3 — Semantic Similarity

Compare embeddings of:

- answer
- context

Low similarity → possible hallucination.

3 Heatmap Debugging

What it is

Heatmap debugging visualizes **which parts of the context influenced the answer**.

It highlights **important tokens or chunks** used by the LLM.

Example:

Context:

Policy waiting period is 30 days.

Policy covers hospitalization.

Heatmap:

waiting period → ● high importance

30 days → ● high importance

hospitalization → ● medium

Colors mean:

Color	Meaning
● Red	Highly relevant
● Yellow	Medium
● Blue	Low relevance

Why Heatmaps Are Useful

They help detect problems like:

- wrong chunk retrieved
- model ignoring correct context
- prompt issues

Tools like Arize Phoenix provide these visualizations.

4 RAG Dashboard

What it is

A **RAG dashboard** shows metrics about the entire RAG system.

Think of it like **monitoring panel for your AI system**.

Example metrics:

Metric	Description
Query count	Number of user queries
Retrieval latency	Time to fetch docs
LLM latency	Response time
Hallucination rate	% incorrect answers
Retrieval accuracy	Retriever performance

Example dashboard view:

RAG System Dashboard

Queries Today: 1240

Average Latency: 1.2s

Hallucination Rate: 5%

Retriever Precision: 87%

Why RAG Dashboard Is Important

It helps detect:

- performance issues
- bad retriever configuration
- increasing hallucination rates

Companies monitor this in production.

5 Query Traces

What is a Query Trace

A **query trace** shows the full pipeline execution for a single query.

Example trace:

User Query:

"What is the policy waiting period?"

Step 1: Embedding

Step 2: Vector DB search

Step 3: Retrieved documents

Step 4: Prompt construction

Step 5: LLM generation

Step 6: Final answer

Each step includes:

- input
- output
- latency
- tokens

Why Query Traces Are Important

They help debug problems like:

Problem	Trace Shows
Wrong answer	LLM step
Bad context	Retriever step
Slow response	Latency step

Tools like:

- Arize Phoenix
- LangSmith

visualize query traces.

Complete LLM Observability Architecture

User Query



FastAPI



Retriever (Vector DB)



Context



Prompt Template



LLM



Answer



Evaluation Layer

- └─ RAGAS metrics
- └─ Hallucination detection
- └─ Heatmap debugging
- └─ Query traces
- └─ RAG dashboard

Simple Summary

Topic	Purpose
RAGAS evaluation	Measure RAG quality
Hallucination detection	Detect incorrect answers
Heatmap debugging	Visualize context importance
RAG dashboard	Monitor system performance
Query traces	Debug individual queries

Simple understanding

RAG development has **3 phases**:

Build RAG



Evaluate RAG



Monitor RAG

These topics belong to the **Evaluate + Monitor stage**.